

PROVA PRÁTICA 1 (P1): PROJETO INTEGRADO DE ALGORITMOS

Engenharia de Software, Estruturas Abstratas de Dados e Algoritmos em Python

INSTRUÇÕES GERAIS

- **Formato de Trabalho:** O projeto pode ser desenvolvido **individualmente** ou em **dupla**.
 - **Linguagem Oficial: Python 3** (código puro; uso permitido da biblioteca padrão: `os`, `sys`, `math`, `pickle`, `struct`, `time`). Vedado o uso de frameworks externos de terceiros.
 - **Identificação:** Nome Completo e RA dos autores devem constar no cabeçalho do código-fonte (`.py`) e na capa/cabeçalho do relatório técnico (`.pdf`).
 - **Submissão:** Envio para douglas.goncalves01@cps.sp.gov.br contendo arquivo compactado (`.zip`) com o código-fonte e o relatório em formato PDF.
 - **Assunto do E-mail:** P1 - Algoritmos Avançados - 022026 - [Nome dos Alunos].
 - **Prazo de Entrega:** Definido pelo professor em sala de aula.
-

1. VISÃO GERAL E FILOSOFIA DA AVALIAÇÃO

A Avaliação Prática 1 (P1) tem como propósito desafiar os estudantes a consolidar, em um software completo e funcional em modo console, todos os conceitos fundamentais, estruturas abstratas de dados e técnicas algorítmicas desenvolvidos ao longo das Aulas 01 a 11 da disciplina.

Escopo Aberto & Autonomia de Criação:

O projeto é de **tema livre**. A equipe possui total autonomia para escolher o domínio do problema a ser implementado (aplicações de telemetria agrícola, logística, classificação de grãos, controle operacional de frotas, automação comercial, simuladores, jogos baseados em terminal, gestão financeira, etc.). Na Seção 3, são apresentadas quatro sugestões temáticas detalhadas como inspiração. Independentemente do tema adotado, **todos os 6 requisitos técnicos obrigatórios descritos a seguir devem ser integralmente atendidos**.

2. REQUISITOS TÉCNICOS OBRIGATÓRIOS (PARTE 1)

Para obter a pontuação máxima, o software desenvolvido deve cobrir com rigor técnico os seguintes pilares:

1. Programação Orientada a Objetos (POO – Aulas 08 e 09):

- Definir ao menos **duas classes** coesas que modelem entidades essenciais do sistema (ex.: `Sensor` e `Talhao`; `Maquina` e `Manutencao`; `Carga` e `Silo`; `Produto` e `Pedido`).
- Construtor (`__init__`) devidamente parametrizado, atributos de instância consistentes e métodos com regras de negócio, validações ou cálculos aplicados.
- Sobrescrita de `__str__` ou `__repr__` para propiciar exibição textual amigável e legível dos objetos no console.

2. TADs e Estruturas de Dados Nativas (Aulas 03, 04 e 05):

- **Listas Dinâmicas:** Armazenamento de séries temporais ou coleções de registros/objetos. É obrigatório o emprego de fatiamento (*slicing*) em pelo menos uma consulta/exibição e de ao menos uma ***list comprehension*** para filtragem ou transformação concisa de dados.
- **Tuplas:** Emprego proposital de tuplas para dados que exigem integridade/imutabilidade estrutural (ex.: coordenadas geoespaciais (`lat`, `lon`), coordenadas de grade (`x`, `y`), faixas operacionais (`min`, `max`) ou registros analíticos imutáveis).
- **Dicionários:** Mapeamento chave-valor para busca direta em tempo constante $O(1)$ por chave/ID primário, ou tabelas de agregação/contagem de frequência.

3. Algoritmo de Busca Sequencial / Linear (Aulas 08 e 09):

- Implementação **manual** de rotina de Busca Linear iterando sobre coleções de dados ou objetos para localizar itens por atributo específico (código, placa, ID, lote, nome, etc.).

- Retorno explícito do registro/índice encontrado ou mensagem clara de registro não localizado.
- *Restrição:* Não é permitido usar apenas o operador `in` sobre coleções rasas ou o método pronto `lista.index()`; a lógica de varredura sequencial deve ser codificada pela dupla.

2. REQUISITOS TÉCNICOS OBRIGATÓRIOS (CONTINUAÇÃO)

4. Algoritmo de Ordenação Manual (Aulas 10 e 11):

- Implementar **manualmente** UM dos algoritmos de ordenação estudados na disciplina:
 - **Opção 1 – Bubblesort:** Deve conter a otimização com *flag* de verificação de trocas para interrupção precoce no melhor caso e limite decrescente de comparações $(n - 1 - i)$; **OU**
 - **Opção 2 – Quicksort:** Deve utilizar a estratégia de divisão e conquista com particionamento (Lomuto ou Hoare), escolha de pivô e chamadas recursivas.
- O algoritmo deve ser aplicado para ordenar dados ou objetos da aplicação por um critério relevante (ex.: nível de criticidade, prioridade de atendimento, data, consumo, valor ou umidade).
- *Restrição Crucial:* **É estritamente vedado o uso de `lista.sort()` ou `sorted()` para o requisito principal.** A mecânica do algoritmo escolhido deve ser integralmente implementada pelos estudantes.

5. Persistência de Dados e I/O de Arquivos (Aulas 07 e 11):

- **Modo Texto (.txt ou .csv):** Leitura e/ou escrita utilizando o gerenciador de contexto (`with open(...)`) para emissão de relatórios consolidados, logs de auditoria ou carregamento inicial de dados tabulares.
- **Modo Binário (.bin ou .pkl):** Persistência do estado das estruturas em arquivo binário (`'wb' / 'rb'`) utilizando o módulo `pickle` OU `struct`, assegurando a preservação e recuperação dos dados entre execuções do programa.

6. Interface de Linha de Comando (CLI) e Clean Code (Aulas 01 e 02):

- Menu interativo em laço contínuo (`while True`), com opções claras de navegação (Cadastrar, Listar/Filtrar, Buscar, Ordenar, Gerar Relatório e Salvar/Sair).
- Tratamento defensivo de exceções com blocos `try/except` para blindar o sistema contra erros de digitação.
- Aderência estrita à PEP 8 e princípios de *Clean Code* (nomes significativos, funções com responsabilidade única).

3. SUGESTÕES TEMÁTICAS PARA INSPIRAÇÃO

A dupla possui total autonomia para escolher o domínio do sistema. Para inspirar a formulação do projeto, apresentamos quatro propostas estruturadas:

Cenário A: Telemetria Agrícola e Manejo Inteligente de Irrigação (IoT no Agro)

- **Contexto:** Monitoramento contínuo de sondas e sensores de solo distribuídos em talhões de citros ou cana-de-açúcar.
- **Classes Sugeridas:** `SensorSolo` (id, umidade, temperatura, bateria) e `Talhao` (código, cultura, sensores associados).
- **Uso de Tuplas:** Coordenadas geográficas ou de grade da sonda (`latitude, longitude`).
- **Uso de Dicionários:** Mapeamento indexado por ID do sensor `{id_sensor: objeto}` ou tabela de alertas acumulados por talhão.
- **Busca Sequencial:** Localizar sensor específico por código de patrimônio ou identificar sondas com leitura anômala.
- **Ordenação (Bubblesort ou Quicksort):** Ordenar os sensores em ordem crescente de umidade do solo para gerar a fila prioritária de acionamento das bombas de irrigação.
- **Persistência:** Exportação de relatório de alertas em texto (`alertas_irrigacao.txt`) e backup binário do estado operacional das sondas (`sensores.pkl` ou `telemetria.bin`).

Cenário B: Gestão e Manutenção Preditiva de Frotas de Máquinas Agrícolas

- **Contexto:** Plataforma operacional para tratores, pulverizadores autopropelidos e colheitadeiras de uma cooperativa agropecuária.
- **Classes Sugeridas:** `MaquinaAgricola` (prefixo, modelo, horímetro atual, consumo médio) e `OrdemServico` (número OS, serviço, prioridade).
- **Uso de Tuplas:** Limites operacionais de tolerância do fabricante (`horimetro_maximo, consumo_alvo`).
- **Uso de Dicionários:** Frota indexada por chassi/código e histórico de abastecimentos agrupados por mês.
- **Busca Sequencial:** Busca manual de máquinas por operador responsável ou status operacional.
- **Ordenação (Bubblesort ou Quicksort):** Ordenar a frota por horas restantes até a revisão preventiva (mais urgente primeiro) ou por consumo médio de combustível por hectare.
- **Persistência:** Carga inicial da frota via arquivo texto/CSV e salvamento binário de todas as ordens de serviço

ativas.

Cenário C: Recepção, Classificação e Alocação de Grãos em Silos

- **Contexto:** Sistema para pesagem, classificação e armazenagem de cargas de soja e milho que chegam aos armazéns gerais.
- **Classes Sugeridas:** *CargaGrao* (placa, produtor, peso líquido, tipo de grão) e *Silo* (capacidade máxima, volume ocupado, grão armazenado).
- **Uso de Tuplas:** Parâmetros analíticos de recepção (*percentual_umidade*, *percentual_impureza*).
- **Uso de Dicionários:** Tabela de precificação/descontos por faixa de qualidade e mapa de alocação de silos.
- **Busca Sequencial:** Localizar carregamento específico pelo número do Romaneio / Nota Fiscal.
- **Ordenação (Bubblesort ou Quicksort):** Ordenar cargas em fila pelo percentual de umidade para organizar a esteira dos secadores industriais.
- **Persistência:** Emissão de romaneio de pesagem em texto e backup da distribuição dos silos em arquivo binário.

Cenário D: Tema Livre (À Escolha dos Estudantes)

- A dupla pode optar por outro tema (ex.: *Vending Machine* inteligente, e-commerce, finanças pessoais, simulador de RPG em terminal, gestão hospitalar/clínica, catálogo de mídias, etc.).
 - **Condição Indispensável:** O sistema precisa contemplar rigorosamente todos os 6 requisitos técnicos da Seção 2 e documentar as decisões no relatório técnico.
-

4. ENTREGÁVEIS OBRIGATÓRIOS

1. **Código-Fonte (.py):** Arquivos limpos, modulares, comentados e funcionais no terminal.
2. **Relatório Técnico (.pdf):** Documento conciso estruturado nas seguintes seções:
 - a) **Identificação:** Título do projeto, integrantes (Nome e RA) e tema adotado.
 - b) **Descrição do Problema e Motivação:** O que a aplicação soluciona e seu valor prático.
 - c) **Modelagem e Arquitetura:** Classes, atributos, métodos e a justificativa do emprego de Listas, Tuplas e Dicionários.
 - d) **Fundamentação dos Algoritmos:** Trechos de código da Busca Sequencial e da Ordenação (Bubblesort ou Quicksort), com análise teórica da complexidade assintótica de tempo ($O(n)$, $O(n^2)$ ou $O(n \log n)$).
 - e) **Mecanismo de Persistência:** Explicação e trechos de código do I/O em modo texto e em modo binário.
 - f) **Casos de Teste e Execução:** Capturas de tela demonstrando o fluxo do menu, busca, ordenação e arquivos gravados.
 - g) **Dificuldades, Bugs Conhecidos e Lições Aprendidas:** Reflexão crítica sobre desafios encontrados e soluções adotadas.

5. RUBRICA DE AVALIAÇÃO (TOTAL: 10,0 PONTOS)

Critério de Avaliação	Aspectos Avaliados	Pontuação
1. Modelagem POO	Ao menos 2 classes com <code>__init__</code> , atributos adequados, métodos com regras de negócio e <code>__str__</code> / <code>__repr__</code> .	2,0 pts
2. TADs e Coleções	Emprego correto das 3 estruturas: <i>slicing</i> e <i>comprehension</i> em listas; tuplas imutáveis; dicionários com acesso $O(1)$ por chave.	2,0 pts
3. Busca Sequencial Manual	Implementação explícita da busca linear com laço, retorno do item/índice e tratamento de registro não encontrado.	1,5 pts
4. Ordenação Manual	Implementação completa de Bubblesort otimizado OU Quicksort (sem <code>.sort()</code>), ordenando dados por critério funcional relevante.	1,5 pts
5. Persistência de Dados	Leitura/gravação em arquivo texto com <code>with open</code> e persistência de dados em arquivo binário (<code>pickle</code> ou <code>struct</code>).	1,5 pts
6. Interface & Clean Code	Menu interativo contínuo no console, tratamento defensivo com <code>try/except</code> para evitar quebras e conformidade com a PEP 8.	0,5 pt
7. Relatório Técnico (PDF)	Documentação clara, análise assintótica dos algoritmos, evidências visuais de execução e honestidade nos bugs/dificuldades.	1,0 pt
Nota Final Máxima		10,0 pts

6. CÓDIGO DE CONDUTA E INTEGRIDADE ACADÊMICA

- **Autonomia e Autoria:** O trabalho prático deve ser fruto exclusivo do esforço intelectual do estudante ou da sua respectiva dupla. Não é permitido solicitar ou aceitar a resolução de código de pessoas externas à equipe.
- **Não Compartilhamento de Código:** É terminantemente proibido repassar códigos-fonte, parciais ou totais, para colegas de outras duplas, bem como obter trechos de soluções alheias. Discutir ideias e estratégias algorítmicas conceituais é encorajado; compartilhar linhas de código caracteriza violação ética grave.
- **Proibição Estrita de Plágio:** O uso de soluções copiadas da web, repositórios de semestres anteriores ou trabalhos de terceiros sem a devida autoria anula sumariamente a avaliação de todos os envolvidos.
- **Arguição Oral Individual Obrigatória:** O professor poderá convocar qualquer estudante ou dupla para uma sabatina e explicação detalhada de qualquer parte do código submetido. A incapacidade de demonstrar domínio e explicar satisfatoriamente as estruturas, métodos e algoritmos entregues resultará na invalidação sumária da nota da referida parte ou do projeto como um todo.

7. DIRETRIZES E RECOMENDAÇÕES PRÁTICAS

- **Projete no papel antes de abrir o editor:** Desenhe os atributos de cada classe, quais tuplas serão necessárias e qual informação servirá como chave do dicionário antes de iniciar a digitação do código.
- **Construa em etapas incrementais:**
 - a) Crie as classes primeiro; teste a instanciação dos objetos no console;
 - b) Implemente o gerenciamento com a lista principal e o dicionário de indexação;
 - c) Adicione a rotina manual de busca sequencial e teste-a isoladamente com diferentes chaves;
 - d) Implemente o algoritmo manual de ordenação (Bubblesort ou Quicksort) e teste a ordenação da lista;
 - e) Desenvolva a persistência em modo texto e em modo binário (`pickle` ou `struct`);
 - f) Por fim, conecte tudo ao menu interativo com tratamento de exceções (`try/except`).
- **Teste de Mesa Rigoroso:** Faça testes manuais nos laços da busca e da ordenação com listas pequenas para garantir que nenhum erro de índice (`IndexError`) ou laço infinito aconteça.
- **Controle de Versão (Git/GitHub):** Recomenda-se enfaticamente que a dupla utilize um repositório no GitHub para registrar as etapas de desenvolvimento, histórico de commits individuais e anotações do projeto.
- **“O ótimo é o inimigo do bom”:** Priorize entregar um sistema estável, legível, robusto e que cumpra 100% dos requisitos obrigatórios antes de tentar planejar funcionalidades periféricas complexas.

Bom trabalho e excelente desenvolvimento! – Prof. Me. Douglas Samuel Gonçalves